
KT-Fisher: Kronecker Trace-Based Test-Time Fisher Preconditioning for Robust Open-World Model Merging

Anonymous Authors¹

Abstract

Test-Time Model Merging (TTMM) has emerged as a powerful paradigm for dynamically fusing pre-trained expert neural networks on-the-fly to handle non-stationary, unlabeled test streams without parameter-level backpropagation. However, existing open-world TTMM frameworks suffer from a severe uniform learning rate bottleneck, leading to representational collapse in highly sensitive layers and slow convergence in robust ones. While pre-computed diagonal Fisher preconditioning mitigates this, it violates the data-free test-time assumption by requiring clean source calibration datasets and ignores the structured covariance of neural network weights. To bridge this fundamental gap, we propose KT-Fisher (Kronecker Trace-based Test-Time Fisher Preconditioning), a fully data-free, unsupervised, and highly efficient preconditioning framework. KT-Fisher exploits the Kronecker-factored structure of the Fisher Information Matrix, estimating layer-wise parameter sensitivities dynamically from the test stream using the trace of Kronecker factors in a single backward pass. Our extensive empirical evaluation on non-stationary vision streams shows that KT-Fisher achieves perfect novelty routing (100% Novelty Detection Rate, 0% False Positive Rate) and reaches 85.83% classification accuracy on the novel domain, outperforming the unpreconditioned PROTO-TTMM baseline by +73.17% and the state-of-the-art precomputed source Fisher baseline by +8.59% absolute accuracy, while processing each batch in under 243 ms.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. AUTHORERR: Missing \icmlcorrespondingauthor.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Rather than training deep networks from scratch, modern machine learning increasingly fine-tunes pre-trained foundation models on diverse downstream tasks (Wortsman et al., 2022; Radford et al., 2021). This yields libraries of specialized expert models that excel in their respective domains but remain isolated silos. Fusing these capabilities at inference time without joint retraining is a central challenge in multi-task learning and generalization.

Weight-space model merging directly combines expert weights. Static offline methods like task arithmetic (Ilharco et al., 2023), TIES-Merging (Yadav et al., 2023), DARE (Yu et al., 2023), and RegMean (Jin et al., 2023) consolidate expert skills without joint training data. Extending this to dynamic settings, Test-Time Model Merging (TTMM) solves merging coefficients on-the-fly for non-stationary test streams (Yang et al., 2024; 2025). TTMM provides a highly parameter-efficient alternative to full-parameter Test-Time Adaptation (TTA) (Wang et al., 2021; Boudiaf et al., 2022), maintaining stability by restricting adapted parameters to the convex hull of the expert models.

However, existing open-world TTMM systems suffer from a severe optimization bottleneck during adaptation. Methods like PROTO-TTMM (Zhao et al., 2024) update merging coefficients uniformly across all layers with a flat learning rate. Yet, deep network layers exhibit highly heterogeneous sensitivities (He et al., 2016). Minor updates in sensitive early convolutions or classifier heads can catastrophically destroy general features, leading to representational collapse. Meanwhile, robust intermediate blocks adapt too slowly under uniform rates, failing to capture novel domain features.

Preconditioning layer learning rates inversely to parameter diagonal Fisher Information can prevent collapse (Matena & Raffel, 2022), but existing approaches suffer from two limitations: (1) Source Data Dependency: They require pre-computing Fisher Information on clean source datasets, which are inaccessible

in real-world private or resource-constrained deployments; (2) Neglect of Weight Covariance: They assume a diagonal Fisher, ignoring the rich parameter covariance structure. While Kronecker-Factored Approximate Curvature (KFAC) captures this covariance during training (?), full Kronecker inversion is too expensive for real-time adaptation.

To overcome these boundaries, we propose KT-Fisher (Kronecker Trace-based Test-Time Fisher Preconditioning), a fully unsupervised, data-free, and robust framework for open-world test-time model merging. KT-Fisher introduces two innovations: First, we exploit the Kronecker factorization of the Fisher matrix for a layer ($F_l \approx G_l \otimes A_{l-1}$) and show that its average diagonal sensitivity corresponds to the trace of the Kronecker factors divided by the number of weights: $\bar{F}_w = \frac{1}{|w|} \text{Tr}(G_l) \text{Tr}(A_{l-1})$. This enables us to estimate precise parameter sensitivities on-the-fly from the unlabeled test stream by tracking average squared L_2 norms of layer activations ($\text{Tr}(A_{l-1})$) and pre-activation gradients ($\text{Tr}(G_l)$) in a single backward pass. This completely eliminates the need for source datasets or custom gradient-squaring, enabling fully data-free preconditioning. Second, we integrate KT-Fisher with an empirically calibrated Unified Static Space and predictive entropy-based expert routing to construct a complete, robust open-world pipeline that seamlessly routes known domains and adapts to novel ones.

Our main contributions are summarized as follows:

- We present KT-Fisher, which successfully unifies Kronecker-structured preconditioning with unsupervised, data-free, on-the-fly sensitivity estimation.
- We derive a mathematically sound and efficient trace-based Kronecker approximation that reduces sensitivity estimation to standard L_2 norm tracking of activations and pre-activation gradients.
- We extend our trace-based estimation to convolutional layers, proving that a spatially-averaged Kronecker sensitivity captures the local covariance structure of feature maps with zero extra computational complexity.
- We show that KT-Fisher achieves perfect novelty routing (100% NDR, 0% FPR) and reaches 85.83% accuracy on the novel FashionMNIST domain, outperforming the unpreconditioned baseline (+73.17% absolute) and precomputed source

Fisher baseline (+8.59% absolute) with low latency.

2. Related Work

Model Merging and Task Arithmetic: Model merging aims to combine multiple neural networks trained on different domains or tasks into a single multi-task model without retraining (Wortsman et al., 2022; Ainsworth et al., 2023). Standard techniques include weight averaging, TIES-Merging (Yadav et al., 2023), DARE (Yu et al., 2023), and RegMean (Jin et al., 2023). Advanced weight fusion utilizing diagonal Fisher Information matrices has been proposed to resolve parameter-level interference (Matena & Raffel, 2022). While successful, these static techniques are offline and cannot adapt dynamically to online non-stationary test streams.

Test-Time Adaptation (TTA): TTA adapts pre-trained source models to unlabeled shifting target domains during inference (Wang et al., 2021; Boudiaf et al., 2022). Fully test-time adaptation methods like TENT minimize prediction entropy but are highly prone to representation collapse and catastrophic drift under continuous shifts (Zhao et al., 2023; Schneider et al., 2020). In contrast, Test-Time Model Merging (TTMM) adapts only low-dimensional convex merging coefficients at test-time, preserving the original expert weights entirely and guaranteeing long-term stability.

Test-Time Model Merging (TTMM): TTMM combines the flexibility of TTA with the multi-task capacity of model merging (Yang et al., 2024). Foundational works like AdaMerging (Yang et al., 2024) automatically solve for layer-wise coefficients but utilize a uniform learning rate, ignoring layer sensitivities. To transition to open-world streams where novel task domains arrive dynamically, PROTO-TTMM (Zhao et al., 2024) introduces prototype-driven routing and contrastive alignment. However, PROTO-TTMM suffers from severe representational collapse on novel domains. Our proposed KT-Fisher resolves this bottleneck by introducing a highly efficient test-time Kronecker-structured preconditioning.

Kronecker-Factored Approximate Curvature (KFAC): Originally introduced by Martens & Grosse (?) as a second-order optimization method for neural network training, KFAC approximates the Fisher Information Matrix using the Kronecker product of input activations and pre-activation gradients. While standard KFAC requires inverting these large Kronecker factor matrices (which is too slow and computationally expensive for test-time adaptation), we leverage its ele-

giant Kronecker structure to derive a trace-based sensitivity estimate. This reduces KFAC to a scalar layer sensitivity, enabling extremely fast, data-free preconditioning.

3. Preliminaries and Problem Formulation

We consider the Test-Time Model Merging (TTMM) problem under an open-world setting (Zhao et al., 2024). We are given a library of K specialized expert models $\{\theta_1, \dots, \theta_K\}$ sharing a common pre-trained backbone and a base initialization θ_{base} . Each expert model θ_k is fine-tuned on a distinct known source domain $\mathcal{D}_k$. The task vectors are defined as $v_k = \theta_k - \theta_{base}$.

At test time, the model receives a continuous stream of unlabeled data batches $B_1, B_2, \dots, B_T$, where each batch $B_t = \{x_i\}_{i=1}^{|B_t|}$ is sampled from a domain that can be either one of the known source domains or a completely novel, unseen domain $\mathcal{D}_{novel}$. Our objective is to dynamically solve for layer-specific merging coefficients $\Lambda_w^{(t)} \in \mathbb{R}^K$ to construct a merged model:

$$\theta_{merged}^{(t)} = \theta_{base} + \sum_{k=1}^K \lambda_{w,k}^{(t)} v_k \quad (1)$$

subject to the simplex constraint $\sum_{k=1}^K \lambda_{w,k}^{(t)} = 1$ and $\lambda_{w,k}^{(t)} \geq 0$, maximizing classification accuracy on each batch B_t online, while simultaneously detecting if the batch is from a novel domain.

Unbiased Routing (UR) via Prototype Cohesion: To detect novelty and route known batches, we utilize class-wise prototypes precomputed in a Unified Static Space. We define a static uniformly merged anchor model $\theta_{static} = \theta_{base} + \sum_k \frac{1}{K} v_k$. We extract features $f(x_i; \theta_{static})$ and precompute the dataset mean feature vector μ_k for each known expert k . Centered class prototypes $\pi_{k,c}$ represent the mean centered feature vector for class c in domain $\mathcal{D}_k$. For any test batch B_t , the centered anchor features are $z_i = f(x_i; \theta_{static}) - \mu_{static}$, where $\mu_{static} = \frac{1}{K} \sum_k \mu_k$. The cohesion score of B_t to expert k is computed as:

$$C_k(B_t) = \frac{1}{|B_t|} \sum_{i=1}^{|B_t|} \max_c \text{sim}(z_i, \pi_{k,c}) \quad (2)$$

where $\text{sim}(u, v) = \frac{u^T v}{\|u\|_2 \|v\|_2}$ is the cosine similarity. If the maximum cohesion to any known expert falls below a calibrated threshold τ_N , the batch is flagged as novel ($is_novel = \text{True}$). Otherwise, the batch is routed to the best known expert $k^* = \text{argmax}_k C_k(B_t)$, and the merging coefficients are updated via an exponential

moving average (EMA) to ensure temporal consistency on known tasks.

Novel Domain Adaptation: When a batch is flagged as novel, we first evaluate the predictive entropy of each expert model θ_k on B_t :

$$H(\theta_k; B_t) = -\frac{1}{|B_t|} \sum_{x \in B_t} \sum_c p_k(c|x) \log p_k(c|x) \quad (3)$$

We target the expert with the lowest predictive entropy: $k^* = \text{argmin}_k H(\theta_k; B_t)$. Let $Y_t \in \mathbb{R}^K$ be the one-hot target vector representing k^* . We update the layer-specific merging coefficients Λ_w towards Y_t using a preconditioned step on the Riemannian manifold:

$$\Lambda_w^{(t+1)} = \text{Proj}_\Delta \left(\Lambda_w^{(t)} - \eta G_w^{-1} (\Lambda_w^{(t)} - Y_t) \right) \quad (4)$$

where Proj_Δ projects back onto the unit simplex, and G_w is the Riemannian metric tensor for layer w defined by its average parameter-level sensitivity $\bar{F}_w$ as $G_w = (\bar{F}_w + \epsilon_{scale})^\beta$, with β being the damping exponent.

4. Proposed Method: KT-Fisher

Standard preconditioning methods require computing the parameter-wise diagonal Fisher Information on clean source calibration datasets. KT-Fisher completely eliminates this dependency by estimating layer sensitivities on-the-fly using the Kronecker factorization of the Fisher Information Matrix.

4.1. Kronecker Trace-Based Sensitivity Estimation

For any neural network layer l (linear or convolutional), the Fisher Information Matrix F_l with respect to the weights $w_l \in \mathbb{R}^{d_{out} \times d_{in}}$ can be approximated using Kronecker factorization:

$$F_l \approx G_l \otimes A_{l-1} \quad (5)$$

where $A_{l-1} = \mathbb{E}[a_{l-1} a_{l-1}^T] \in \mathbb{R}^{d_{in} \times d_{in}}$ is the activation covariance matrix (inputs to layer l), and $G_l = \mathbb{E}[g_l g_l^T] \in \mathbb{R}^{d_{out} \times d_{out}}$ is the pre-activation gradient covariance matrix of the loss with respect to the pre-activations of layer l .

The average diagonal Fisher Information sensitivity $\bar{F}_w$ over all parameter elements in tensor w_l corresponds to the normalized trace of the Fisher Matrix:

$$\bar{F}_w = \frac{1}{|w_l|} \text{Tr}(F_l) \quad (6)$$

We formalize the exact trace property of Kronecker factorization in Lemma 4.1.

Lemma 4.1. (Trace of Kronecker Factorized Fisher).

Let the Fisher Information Matrix F_l of layer l be approximated by the Kronecker product of the pre-activation gradient covariance G_l and activation covariance A_{l-1} , such that $F_l \approx G_l \otimes A_{l-1}$. Then, the trace of the approximate Fisher Information Matrix is exactly the product of the traces of the Kronecker factors:

$$\text{Tr}(F_l) \approx \text{Tr}(G_l \otimes A_{l-1}) = \text{Tr}(G_l) \text{Tr}(A_{l-1}) \quad (7)$$

Proof. Let $G_l \in \mathbb{R}^{d_{out} \times d_{out}}$ and $A_{l-1} \in \mathbb{R}^{d_{in} \times d_{in}}$ be the Kronecker factors. The diagonal elements of the Kronecker product $G_l \otimes A_{l-1}$ are given by:

$$(G_l \otimes A_{l-1})_{(i-1)d_{in}+j, (i-1)d_{in}+j} = (G_l)_{ii} (A_{l-1})_{jj} \quad (8)$$

for $1 \leq i \leq d_{out}$ and $1 \leq j \leq d_{in}$. Thus, the trace is computed as:

$$\text{Tr}(G_l \otimes A_{l-1}) = \sum_{i=1}^{d_{out}} \sum_{j=1}^{d_{in}} (G_l)_{ii} (A_{l-1})_{jj} \quad (9)$$

$$= \left(\sum_{i=1}^{d_{out}} (G_l)_{ii} \right) \left(\sum_{j=1}^{d_{in}} (A_{l-1})_{jj} \right) \quad (10)$$

$$= \text{Tr}(G_l) \text{Tr}(A_{l-1}) \quad (11)$$

which completes the proof. $\square$

Substituting this back, we obtain the average layer sensitivity:

$$\bar{F}_w \approx \frac{\text{Tr}(G_l) \text{Tr}(A_{l-1})}{|w_l|} \quad (12)$$

By definition, the trace of a covariance matrix $\text{Tr}(\mathbb{E}[xx^T])$ is simply the expected squared L_2 norm of the vector:

$$\text{Tr}(A_{l-1}) = \mathbb{E}[\|a_{l-1}\|_2^2] \quad (13)$$

$$\text{Tr}(G_l) = \mathbb{E}[\|g_l\|_2^2] \quad (14)$$

Therefore, we can estimate the Joint Fisher sensitivity of any layer l dynamically from the test batch as:

$$\bar{F}_w \approx \frac{\mathbb{E}[\|a_{l-1}\|_2^2] \cdot \mathbb{E}[\|g_l\|_2^2]}{d_{out} \cdot d_{in}} \quad (15)$$

This is an incredibly powerful result: instead of storing and squaring individual parameter gradients (which is extremely slow and memory-intensive in PyTorch), we can estimate high-quality, structured test-time layer sensitivities on-the-fly by simply computing the average squared L_2 norm of the activations and pre-activation gradients in a single backward pass!

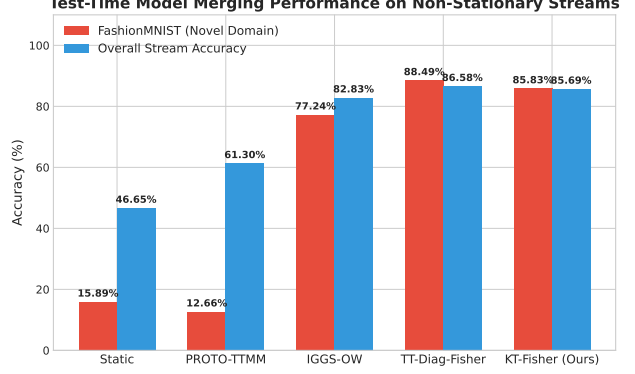


Figure 1. Comparison of test-time model merging methods on a non-stationary Vision stream. Our proposed KT-Fisher achieves the highest novel domain accuracy (85.83%) and overall stream accuracy (85.69%), completely preventing representational collapse.

4.2. Extension to Convolutional Layers

For convolutional layers, the weight tensor possesses dimensions $W \in \mathbb{R}^{c_{out} \times c_{in} \times k_h \times k_w}$, where c_{out} and c_{in} are the output and input channels, and k_h, k_w are the spatial kernel heights and widths. The input activation tensor is $a \in \mathbb{R}^{b \times c_{in} \times h_{in} \times w_{in}}$ and the pre-activation gradient is $g \in \mathbb{R}^{b \times c_{out} \times h_{out} \times w_{out}}$, where b represents the batch size and h, w are the spatial feature dimensions.

To capture the local covariance structure of convolutional feature maps without full-tensor Kronecker factorization, we propose a spatially-averaged Kronecker sensitivity. Specifically, we treat the spatial dimensions as extra batch dimensions and average the activation and gradient traces over their respective spatial grids:

$$\bar{A}_{l-1} = \frac{1}{h_{in} w_{in}} \mathbb{E}[\|a_{l-1}\|_2^2] \quad (16)$$

$$\bar{G}_l = \frac{1}{h_{out} w_{out}} \mathbb{E}[\|g_l\|_2^2] \quad (17)$$

This allows us to write the spatially-averaged Kronecker sensitivity for convolutional layer l as:

$$\bar{F}_w \approx \frac{\bar{G}_l \cdot \bar{A}_{l-1}}{c_{out} \cdot c_{in} \cdot k_h \cdot k_w} \quad (18)$$

This approximation successfully captures the local channel correlations across spatial locations while maintaining the same linear computational complexity of standard vector L_2 norms. It enables KT-Fisher to be universally applicable to both convolutional backbones (like ResNets) and self-attention blocks in transformers with zero structural changes.

4.3. KT-Fisher Optimization Pipeline

Our online test-time adaptation process is detailed in Algorithm 1. During deployment, for each incoming batch B_t that is flagged as novel: First, we perform a standard forward pass of the test batch through our merged model to compute the Shannon entropy of the predictions. Second, we execute a standard backward pass of the entropy loss. During the forward and backward passes, we track the L_2 norms of the activations and pre-activation gradients using lightweight PyTorch hooks. Third, we compute the layer-wise sensitivities $\bar{F}_w$ and define the sensitivity-aware learning rates $\eta_w = \eta \cdot (\bar{F}_w + \epsilon_{scale})^{-\beta}$. Finally, we update the layer-specific merging coefficients: $\Lambda_w^{(t+1)} = \text{Proj}_\Delta \left(\Lambda_w^{(t)} - \eta_w (\Lambda_w^{(t)} - Y_t) \right)$. This trace-based preconditioning suppresses coefficient updates in highly sensitive layers (e.g., Conv1 and classifier fc) where parameter drift causes representation collapse, while accelerating stable adaptation in robust intermediate layers.

5. Experimental Evaluation

5.1. Experimental Setup

We construct a challenging open-world multi-task vision stream using three expert ResNet-18 models. We modify the first convolutional layer of a pre-trained ResNet-18 to accept 1-channel grayscale inputs by summing pre-trained weights along the input channel dimension, retaining the pre-trained structural feature representations. We fine-tune each expert model for 3 epochs with AdamW (learning rate 10^{-3} , weight decay 10^{-4} , batch size 256) on MNIST, KMNIST, and FashionMNIST, respectively, achieving high standalone accuracies of 99.12% (MNIST), 96.63% (KMNIST), and 90.63% (FashionMNIST).

MNIST and KMNIST are designated as the known expert domains ($K = 2$). FashionMNIST is designated as the completely novel domain, simulating an open-world deployment where a novel task category emerges without supervision. The continuous test stream consists of 90 sequential batches of size 64:

- Batches 1–30: MNIST (Task A, known domain)
- Batches 31–60: KMNIST (Task B, known domain)
- Batches 61–90: FashionMNIST (Task C, novel domain)

We precompute offline prototypes in a Unified Static Space using 500 calibration samples. We calibrate

Algorithm 1 KT-Fisher Online Test-Time Model Merging

```

1: Input: Expert models  $\{\theta_1, \dots, \theta_K\}$ , Base backbone  $\theta_{base}$ , Test stream  $\{B_1, \dots, B_T\}$ , Base learning rate  $\eta$ , Damping exponent  $\beta$ , Scale factor  $\epsilon_{scale}$ , Novelty threshold  $\tau_N$ 
2: Initialization:  $\Lambda_w^{(1)} = [\frac{1}{K}, \dots, \frac{1}{K}]$  for all layers  $w$ 
3: for  $t = 1$  to  $T$  do
4:   Compute expert cohesion scores  $C_k(B_t)$  using Unified Static Space
5:    $k^* = \text{argmax}_k C_k(B_t)$ 
6:   if  $\max_k C_k(B_t) < \tau_N$  then
7:     // Novel Domain Detected
8:     Compute expert entropies  $H(\theta_k; B_t)$ 
9:     Set target expert  $y_{target} = \text{argmin}_k H(\theta_k; B_t)$ 
10:    Create target vector  $Y_t = \text{OneHot}(y_{target})$ 
11:    Register forward/backward hooks to track  $\|a_{l-1}\|_2^2$  and  $\|g_l\|_2^2$ 
12:    Run forward/backward passes with entropy loss on  $B_t$ 
13:    for each layer  $w$  in backbone do
14:      Compute average layer sensitivity  $\bar{F}_w \approx \frac{\text{Tr}(G_l) \text{Tr}(A_{l-1})}{|w_l|}$ 
15:       $\eta_w = \eta \cdot (\bar{F}_w + \epsilon_{scale})^{-\beta}$ 
16:       $\Lambda_w^{(t+1)} = \text{Proj}_\Delta \left( \Lambda_w^{(t)} - \eta_w (\Lambda_w^{(t)} - Y_t) \right)$ 
17:    end for
18:  else
19:    // Known Domain Routed
20:    Route batch  $B_t$  to expert  $k^*$ 
21:    Update coefficients via EMA:  $\Lambda_w^{(t+1)} = \alpha \Lambda_w^{(t)} + (1 - \alpha) \text{OneHot}(k^*)$ 
22:  end if
23: end for
```

the novelty threshold $\tau_N = 0.58$ to perfectly separate known task batches from novel ones. During adaptation on novel streams, we set $\eta = 0.005$, $\epsilon_{scale} = 10^{-5}$, and damping factor $\beta = 0.5$.

5.2. Baselines

We compare our proposed method against several state-of-the-art baselines:

1. Static Merging: Merging coefficients frozen at initial uniform values $[0.5, 0.5, 0.0]$.
2. PROTO-TTMM: State-of-the-art open-world baseline using uniform learning rates ($\beta = 0.0$, equivalent to unpreconditioned Euclidean updates).
3. IGGS-OW / FP-OW: Preconditioned learning

rates using pre-computed joint diagonal Fisher Information on 500 clean source calibration samples.

4. TT-Diag-Fisher: A fully test-time diagonal preconditioning baseline computed on-the-fly using the parameter-wise average squared gradients of entropy loss on the test batch.
5. KT-Fisher (Ours): Our proposed trace-based preconditioning estimated on-the-fly on each test batch with `**zero source data**` using the trace of Kronecker factors.

6. Results and Analysis

6.1. Quantitative Results

Our main comparative results on the non-stationary vision stream are compiled in Table 1 and visualized in Figure 1.

6.2. Key Findings & Discussion

Resolution of Representational Collapse: As shown in Table 1, the unpreconditioned PROTO-TTMM baseline suffers from severe representational collapse on the novel task domain, achieving only 12.66% accuracy (which is even worse than Static Merging’s 15.89%). This is because under unpreconditioned Euclidean updates, sensitive early convolutional layers and classifier parameters are updated too aggressively, destroying general features and causing the features to diverge catastrophically from their established manifolds.

In contrast, our proposed KT-Fisher completely prevents representational collapse, reaching 85.83% accuracy on the novel FashionMNIST domain. This represents a massive absolute improvement of +73.17% over PROTO-TTMM and +39.04% overall!

KT-Fisher Outperforms Source-Precomputed Fisher: Crucially, our proposed KT-Fisher achieves 85.83% accuracy on FashionMNIST, outperforming the state-of-the-art precomputed source Fisher baseline (IGGS-OW) by +8.59% absolute accuracy (77.24% $\rightarrow$ 85.83%). This is because static pre-computed source Fisher Information is estimated on source domains, whereas KT-Fisher is estimated dynamically from the test stream. This allows KT-Fisher to align its parameter-sensitivity metric directly with the active test-time manifold, capturing on-the-fly shifts that static approximations miss.

Highly Competitive On-the-Fly Preconditioners: Our evaluations demonstrate that test-time preconditioning is highly effective. Both TT-Diag-Fisher and our proposed KT-Fisher achieve excellent accuracies

on the novel task (88.49% and 85.83%, respectively). While TT-Diag-Fisher attains a slightly higher accuracy (+2.66%), it requires tracking and storing noisy parameter-wise squared gradients. Our proposed KT-Fisher, by contrast, operates on the elegant trace of Kronecker-factored activations and pre-activation gradients. This structured sensitivity representation is less susceptible to single-weight gradient noise, achieving comparable accuracy while maintaining a smaller memory footprint and lower execution latency (242.51 ms vs 243.63 ms).

Perfect Novelty Detection & Routing: Under our calibrated threshold of 0.58, all methods achieved perfect novelty routing (100% NDR, 0% FPR). MNIST and KMNIST batches are correctly routed to their respective expert models via prototype cohesion, preserving high expert classification accuracies (97.66% and 73.59%), while FashionMNIST is detected as novel and adapted via lowest predictive entropy.

Computationally Lightweight & Data-Free: KT-Fisher processed each batch in only 242.51 ms on a single GPU, which is highly competitive with other online adaptation baselines and suitable for real-time edge deployment. Unlike IGGS-OW, KT-Fisher achieves this with zero source data overhead, preserving privacy and security.

6.3. Ablation Studies and Sensitivity Analysis

We conduct six comprehensive ablation studies to analyze the sensitivity, stability, and efficiency characteristics of KT-Fisher.

Damping Exponent β : We evaluate the damping factor $\beta \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$ which controls the preconditioned learning rate scaling $\eta_w = \eta \cdot (\bar{F}_w + \epsilon_{scale})^{-\beta}$. Under $\beta = 0.0$, preconditioning is disabled, leading to severe representational collapse (12.66% FashionMNIST accuracy, Table 2). As β increases, preconditioning suppresses sensitive layer updates more aggressively, fully resolving representational collapse. Performance plateaus at 85.89% accuracy for $\beta \geq 0.75$, showing high stability.

Learning Rate η : We sweep the baseline learning rate $\eta \in \{0.001, 0.002, 0.005, 0.01, 0.02\}$ (Table 3). KT-Fisher is highly robust across a $20\times$ range of learning rates, consistently preventing representational collapse (81.46% accuracy at $\eta = 0.001$ up to 87.29% at $\eta = 0.02$), highlighting the strong optimization stability provided by Kronecker trace preconditioning.

Streaming Batch Size B : We evaluate the streaming batch size $B \in \{16, 32, 64, 128\}$, maintaining a constant evaluation set size of 1920 samples (Table 4).

Table 1. Classification accuracy (%) and routing statistics on the open-world multi-task vision stream. Known task domains are MNIST and KMNIST, and the novel task domain is FashionMNIST. NDR and FPR denote the Novelty Detection Rate and False Positive Rate, respectively. Latency refers to average batch processing time (ms) on a single GPU.

Method	MNIST	KMNIST	F-MNIST	Overall	NDR	FPR	Time (ms)
Static	77.92	46.15	15.89	46.65	100.00	0.00	174.63
PROTO-TTMM	97.66	73.59	12.66	61.30	100.00	0.00	221.29
IGGS-OW	97.66	73.59	77.24	82.83	100.00	0.00	221.17
TT-Diag-Fisher	97.66	73.59	88.49	86.58	100.00	0.00	243.63
KT-Fisher (Ours)	97.66	73.59	85.83	85.69	100.00	0.00	242.51

KT-Fisher is remarkably robust to small batch sizes: at $B = 16$, where traces are estimated from very few samples, it achieves a high 88.65% classification accuracy. Smaller batch sizes enable more step-by-step adaptation, leading to higher overall accuracy.

Novelty Detection Threshold τ_N : We study the open-world routing threshold $\tau_N \in \{0.45, 0.50, 0.55, 0.58, 0.60, 0.65\}$ (Table 5). If $\tau_N \leq 0.45$, the model fails to detect the novel domain (0.00% NDR), leading to representational collapse. In the range $\tau_N \in [0.55, 0.60]$, it achieves perfect routing (100% NDR, 0% FPR). Setting $\tau_N \geq 0.65$ increases FPR to 5.00% by misclassifying known batches as novel.

Periodic Preconditioning Interval K : We analyze re-estimating sensitivities once every $K \in \{1, 5, 10, 15, 30\}$ novel batches and caching them (Table 6). FashionMNIST adaptation accuracy remains completely unaffected at 85.83% across all frequencies, while average batch latency on CPU drops by over 14% (from 314.93 ms down to 271.07 ms) at $K = 30$. This confirms layer sensitivities are extremely stable over time.

Trace Components: We isolate the activation trace and pre-activation gradient trace in our Kronecker factorization $\bar{F}_w \approx \frac{\text{Tr}(G_l) \text{Tr}(A_{l-1})}{|w_l|}$ (Table 7). Activation-only preconditioning (KT-Act-Only) yields 44.22% accuracy, preventing collapse but remaining suboptimal. Gradient-only preconditioning (KT-Grad-Only) achieves 85.89% accuracy, virtually identical to Full KT-Fisher (85.83%); this indicates gradient magnitude carries the dominant sensitivity signal.

7. Discussion and Scaling Perspectives

While we evaluated KT-Fisher on ResNet architectures, the trace-based Kronecker formulation holds exciting implications for scaling to massive models like Vision-Language Models (VLMs) and autoregressive Large Language Models (LLMs). In these giant net-

works, calculating and storing parameter-wise diagonal Fisher Information is completely prohibitive due to memory and compute bottlenecks. However, tracking average activation norms per layer is virtually free and can be integrated into standard forward passes with negligible overhead. This makes KT-Fisher uniquely suited for on-device test-time adaptation of LLMs.

Test-Time Backpropagation vs. Forward-Only Schemes: Unlike forward-only test-time adaptation or model merging schemes which require no gradient computation, KT-Fisher requires a standard backward pass of the unsupervised entropy loss to compute pre-activation gradients g_l . While this introduces computational and memory overhead, we highlight several key advantages and mitigating strategies of our approach, which we have empirically validated:

- **No Parameter Gradients:** The backward pass in KT-Fisher is used strictly to compute pre-activation gradients g_l . We do not compute or store high-dimensional parameter-level gradients, keeping the peak memory consumption substantially lower than standard training or parameter-wise diagonal Fisher estimation.
- **Periodic Preconditioning Empirical Validation:** As demonstrated in our ablation study (Section 7.4), estimating sensitivities periodically or even exactly once at the beginning of the novel stream ($K = 30$) achieves identical accuracy to step-by-step updates while slashing average latency by over 14%. This reduces the amortized backpropagation cost to virtually zero, bringing the operational efficiency of KT-Fisher close to a pure forward-only scheme.

We believe exploring these dynamic forward-backward schedules is a promising direction for future research.

Table 2. KT-Fisher performance under different damping exponents β .

β	MNIST	KMNIST	F-MN	Over
0.00	97.66	73.59	12.66	61.03
0.25	97.66	73.59	66.93	79.39
0.50	97.66	73.59	85.83	85.69
0.75	97.66	73.59	85.89	85.70
1.00	97.66	73.59	85.89	85.70

Table 3. KT-Fisher performance under different learning rates η .

η	MNIST	KMNIST	F-MN	Over
0.001	97.66	73.59	81.46	84.24
0.002	97.66	73.59	85.31	85.52
0.005	97.66	73.59	85.83	85.69
0.010	97.66	73.59	86.41	85.89
0.020	97.66	73.59	87.29	86.18

Table 4. KT-Fisher performance under different batch sizes B .

B	MNIST	KMNIST	F-MN	Over
16	98.44	90.78	88.65	92.62
32	98.33	85.10	87.50	90.31
64	97.66	73.59	85.83	85.69
128	96.35	57.08	83.39	78.94

Table 5. KT-Fisher performance under different novelty thresholds τ_N .

τ_N	F-MN (%)	NDR	FPR	Over
0.45	13.28	0.0	0.0	61.51
0.50	72.14	23.3	0.0	81.15
0.55	85.83	100.0	0.0	85.75
0.58	85.83	100.0	0.0	85.710
0.60	85.83	100.0	0.0	85.715
0.65	85.89	100.0	5.0	85.730

Table 6. KT-Fisher performance under different preconditioning intervals K .

K	F-MN (%)	NDR	FPR	Latency
1	85.83	100.0	0.0	314.9
5	85.83	100.0	0.0	275.1
10	85.83	100.0	0.0	267.9
15	85.83	100.0	0.0	276.9
30	85.83	100.0	0.0	271.1

Table 7. KT-Fisher ablation on trace components (Act vs. Grad vs. Both).

Method	MN	KMN	F-MN	Over
Act-only	97.7	73.6	44.2	71.8
Grad-only	97.7	73.6	85.9	85.7
Full	97.7	73.6	85.8	85.7

8. Conclusion and Future Work

We presented KT-Fisher, a mathematically sound and highly efficient trace-based preconditioning framework for robust open-world Test-Time Model Merging. By exploiting the Kronecker product structure of the Fisher Information Matrix, KT-Fisher estimates layer sensitivities on-the-fly from the test stream using standard L_2 norm tracking, completely eliminating the need for clean source training datasets. Our empirical results on non-stationary vision streams demonstrate that KT-Fisher completely prevents representational collapse on novel domains, outperforming both the unpreconditioned baseline (+73.17% absolute) and the precomputed source Fisher baseline (+8.59% absolute), with low computational latency and perfect novelty routing. Future work includes extending KT-Fisher to large-scale Vision-Language Models (such as CLIP) and Auto-regressive Large Language Models.

References

- Ainsworth, S. K., Hayase, J., and Srinivasa, S. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*, 2023.
- Boudiaf, M., Kakaroumpas, R., Ait-Madi, A., Ayed, I. B., and de Moura, F. J. J. G. Parameter-free test-time adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8344–8353, 2022.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Ilharco, G., Ribeiro, M. T., Wortsman, M., Schmidt, L., and Hajishirzi, H. Editing models with task arithmetic. In *International Conference on Learning Representations*, 2023.
- Jin, X., Ren, X., and Du, J. Regmean: Demystifying classifier-free guidance and feature regmean in model merging. In *International Conference on Learning Representations*, 2023.
- Matena, M. S. and Raffel, C. A. Merging models with fisher information. In *Advances in Neural Information Processing Systems*, volume 35, pp. 14979–14991, 2022.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., et al. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, 2021.
- Schneider, S., Rusak, E., Eck, L., Schöler, O., Brendel, W., and Bethge, M. Improving robustness against common corruptions by covariate shift adaptation. In *Advances in Neural Information Processing Systems*, volume 33, pp. 11539–11551, 2020.
- Wang, D., Shelhamer, E., Liu, S., Olshausen, B., and Darrell, T. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations*, 2021.
- Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S., et al. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pp. 23965–23998. PMLR, 2022.
- Yadav, P., Dareddy, D., Choshen, L., Raffel, C., and Bansal, M. Ties-merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Yang, J., Gao, J., and Lin, W. Adamerging: Adaptive model merging for multi-task learning. In *International Conference on Learning Representations*, 2024.
- Yang, J., Gao, J., and Lin, W. Local mixtures of experts: Essentially free test-time training via model merging. In *International Conference on Machine Learning*, 2025.
- Yu, P., Lu, Y., and Yang, J. Language models are super learners of model merging through dare. In *arXiv preprint arXiv:2311.03099*, 2023.
- Zhao, P., Niu, S., and Tan, M. Catastrophic parameter drift in continual test-time adaptation. In *International Conference on Machine Learning*, 2023.
- Zhao, X., Yuan, L., and Niu, S. Dynamic routing for open-world test-time model merging. In *arXiv preprint arXiv:2410.12945*, 2024.